EEF-205E

Introduction to Logıc Design

HW3

Mehmet Ünlü-040230253

Part I

x	4	2	0	A	B	C	M
0	0	0	1	0	0	1	0
0	0	1	2	0	1	0	0
0	1	0	3	0	1	1	0
0	1	1	4	1	0	0	1
1	0	0	2	0	1	0	0
1	0	1	3	0	1	1	1
1	1	0	4	1	0	0	1
1	1	1	5	1	0	1	1

$A = \sum_m (3, 6, 7)$

$B = \sum_m (1, 2, 4, 5)$

$C = \sum_m (0, 2, 5, 7)$

$M = \sum_m (3, 5, 6, 7)$

A

2 \ xy	00	01	11	10
0			1	
1		1	1	1

$A = xy + y2$

B

2 \ xy	00	01	11	10
0		1		1
1	1			1

$B = xy' + y'2 + x'y2'$

C

2 \ xy	00	01	11	10
0	1			1
1		1	1	

$C = y'2' + y2$

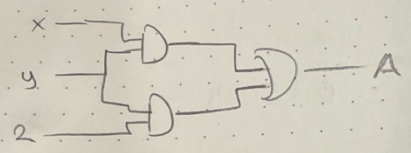
M

2 \ xy	00	01	11	10
0			1	
1		1	1	1

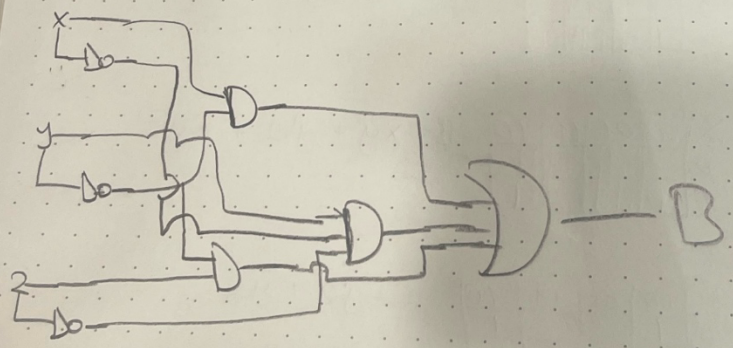
$M = xy + x2 + y2$

BAYKAR

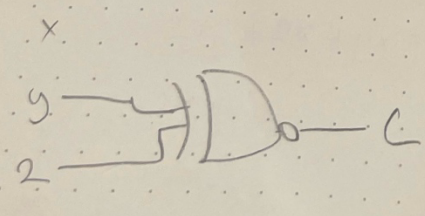
A = xy + yz

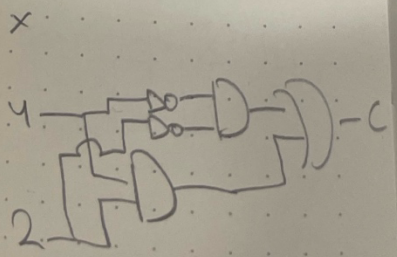


B = xy' + y'z + x'yz'

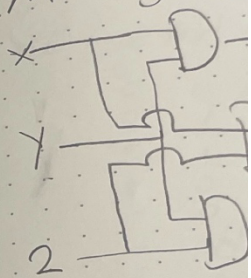


C = y'z' + yz



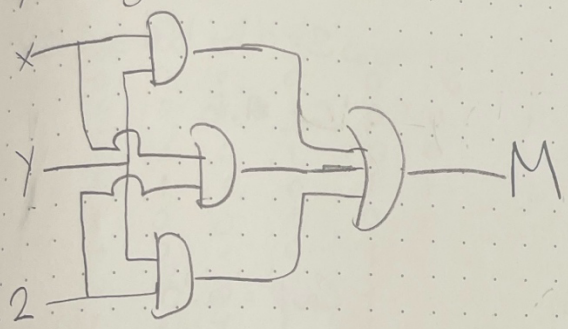


M = xy + y



BAYKAR

$M = xy + y2 + x2$

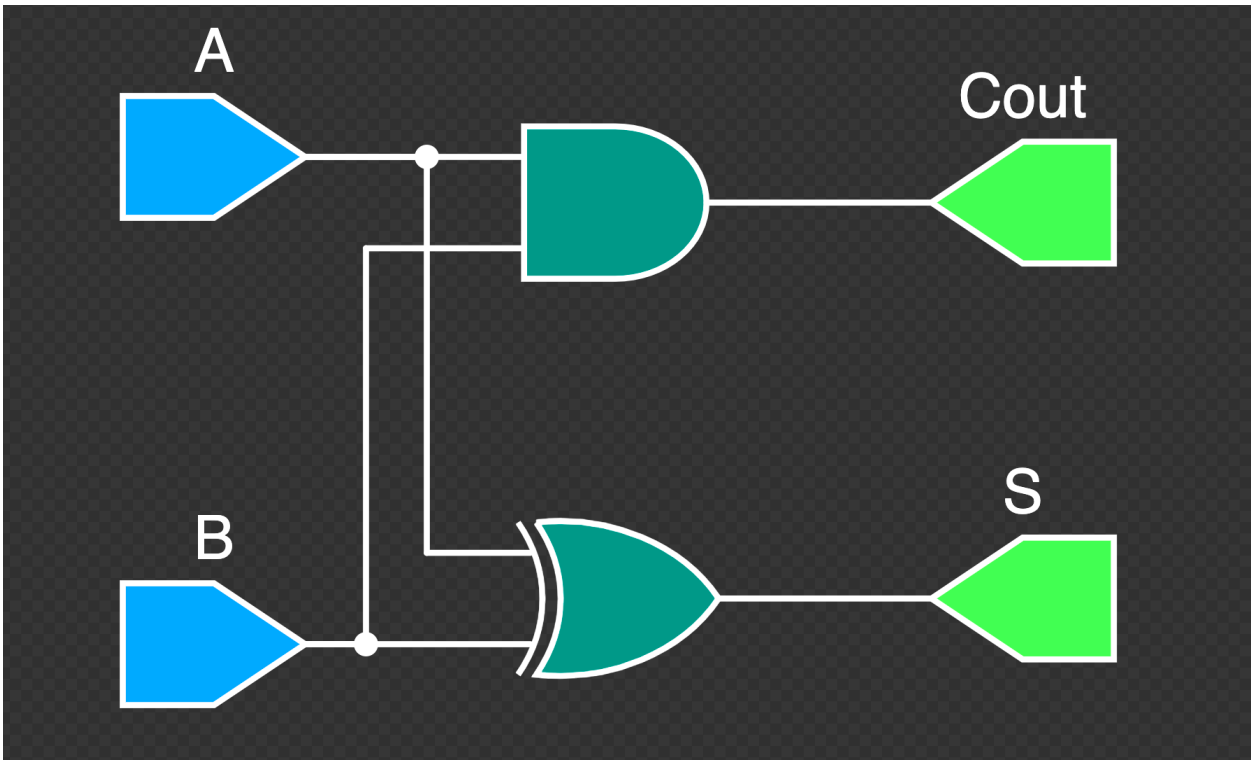


BAYKAR

Part 2

HA

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity HALF_ADDER is
    Port ( A : in  STD_LOGIC;
        B : in  STD_LOGIC;
        S : out STD_LOGIC;
        Cout : out STD_LOGIC);
end HALF_ADDER;

architecture Dataflow of HALF_ADDER is
begin
    -- Dataflow Modeling
    S <= A xor B;
    Cout <= A and B;
end Dataflow;
```



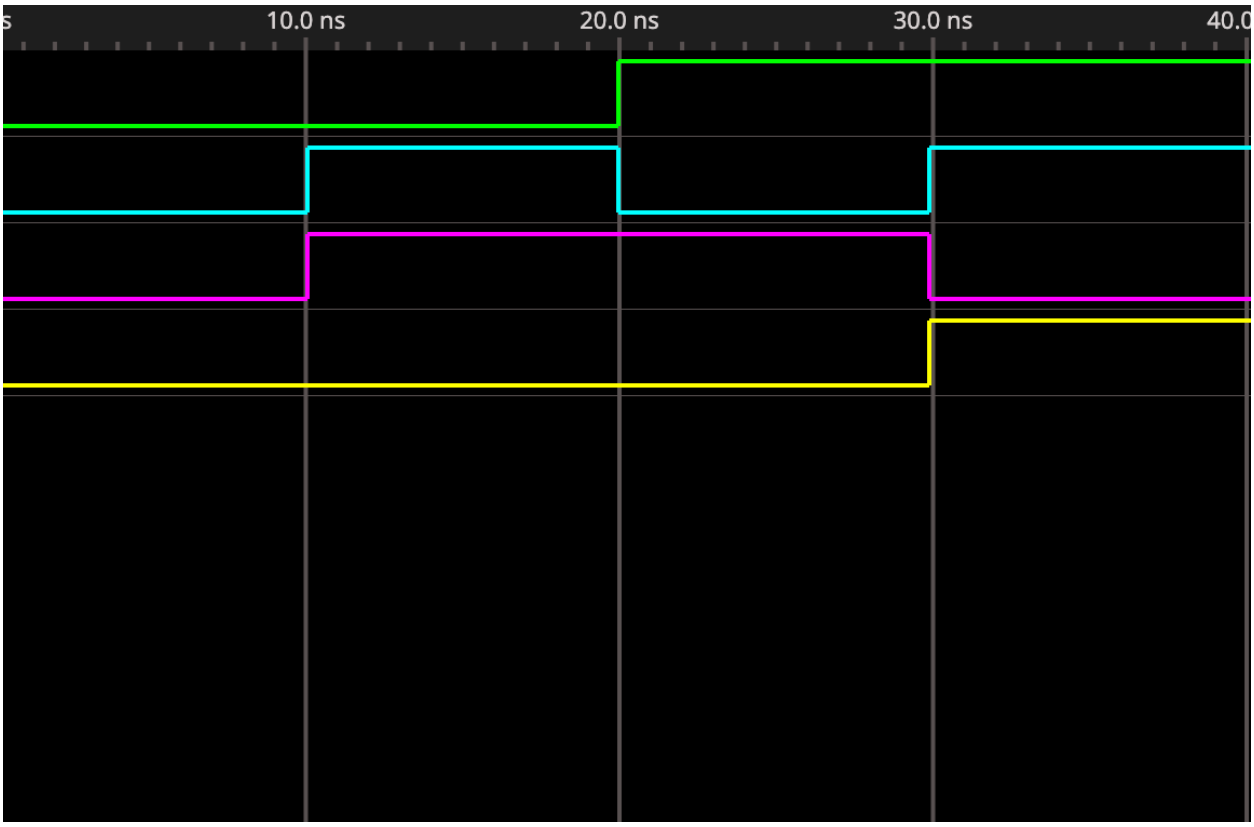
HA Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Sadece simülasyon amacıyla kullanılır, sentezlenemez.
entity HALF_ADDER_TB is
end HALF_ADDER_TB;

architecture Behavioral of HALF_ADDER_TB is
```

```
    -- Test edilecek bileşenin (HA) tanımlanması
    component HALF_ADDER
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           S : out STD_LOGIC; -- Sum
           Cout : out STD_LOGIC); -- Carry Out
end component;

    -- Sinyal tanımlamaları (HA'nın giriş/çıkışlarını bağlamak için)
signal A_TB, B_TB : STD_LOGIC := '0'; -- Başlangıçta 0 atanır
signal S_TB, Cout_TB : STD_LOGIC;

    -- Simülasyon için zaman sabiti
constant Clock_Period : time := 10 ns;

begin
```

```
-- Test edilecek bileşenin bağlanması (Instance)
uut: HALF_ADDER
port map (
    A => A_TB,
    B => B_TB,
    S => S_TB,
    Cout => Cout_TB
);

-- Giriş Vektörlerini Üreten İşlem (Stimulus)
stim_proc: process
begin
    -- Başlangıç (A=0, B=0)
    wait for Clock_Period;

    -- Test 1: A=0, B=1
    B_TB <= '1';
    wait for Clock_Period;
```

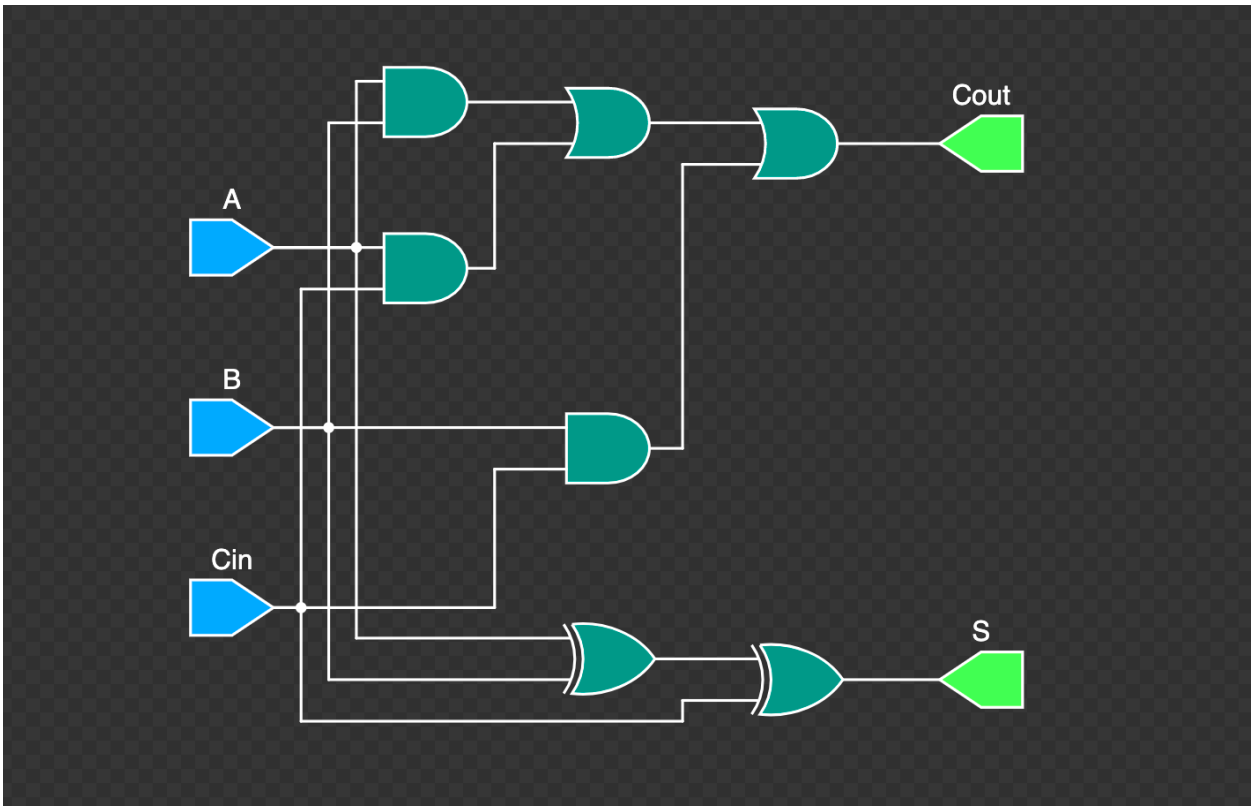
```
        -- Test 2: A=1, B=0
        A_TB <= '1';
        B_TB <= '0';
        wait for Clock_Period;

        -- Test 3: A=1, B=1
        B_TB <= '1';
        wait for Clock_Period;

        -- Simülasyonu sonlandır
        wait;
    end process;
end Behavioral;
```



FA

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity FULL_ADDER is
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           Cin : in  STD_LOGIC; -- Carry In
```

```
        S : out STD_LOGIC; -- Sum
        Cout : out STD_LOGIC); -- Carry Out
end FULL_ADDER;

architecture Dataflow of FULL_ADDER is
begin
    -- Dataflow Modeling
    -- Toplam (S) için denklem: A XOR B XOR Cin
    S <= A xor B xor Cin;

    -- Elde (Cout) için sadeleştirilmiş denklem: (A AND B) OR (A AND Cin) OR (B AND Cin)
    Cout <= (A and B) or (A and Cin) or (B and Cin);
end Dataflow;
```



FA Testbench

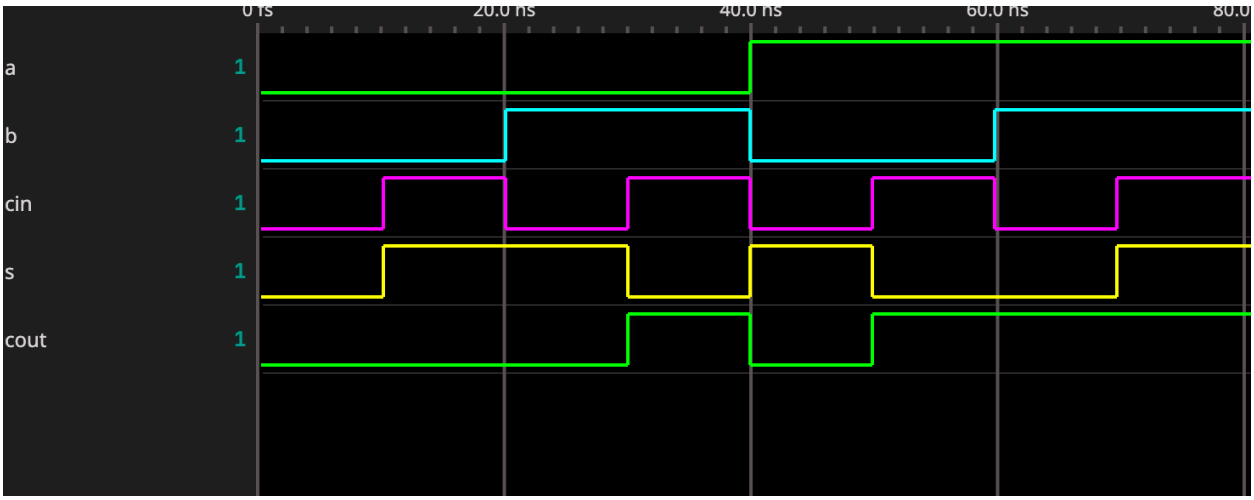
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Sadece simülasyon amacıyla kullanılır, sentezlenemez.
entity HALF_ADDER_TB is
end HALF_ADDER_TB;

architecture Behavioral of HALF_ADDER_TB is
```

```
    -- Test edilecek bileşenin (HA) tanımlanması
    component HALF_ADDER
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           S : out STD_LOGIC; -- Sum
           Cout : out STD_LOGIC); -- Carry Out
end component;

    -- Sinyal tanımlamaları (HA'nın giriş/çıkışlarını bağlamak için)
signal A_TB, B_TB : STD_LOGIC := '0'; -- Başlangıçta 0 atanır
signal S_TB, Cout_TB : STD_LOGIC;

    -- Simülasyon için zaman sabiti
constant Clock_Period : time := 10 ns;

begin
```

```
-- Test edilecek bileşenin bağlanması (Instance)
uut: HALF_ADDER
port map (
    A => A_TB,
    B => B_TB,
    S => S_TB,
    Cout => Cout_TB
);

-- Giriş Vektörlerini Üreten İşlem (Stimulus)
stim_proc: process
begin
    -- Başlangıç (A=0, B=0)
    wait for Clock_Period;

    -- Test 1: A=0, B=1
    B_TB <= '1';
    wait for Clock_Period;
```

```
    -- Test 2: A=1, B=0
    A_TB <= '1';
    B_TB <= '0';
    wait for Clock_Period;

    -- Test 3: A=1, B=1
    B_TB <= '1';
    wait for Clock_Period;

    -- Simülasyonu sonlandır
    wait;
end process;
```

end Behavioral;



RCA

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4BIT_STRUCTURAL is

Port (A : in STD_LOGIC_VECTOR (3 downto 0); -- 4-bit Giriş A

B : in STD_LOGIC_VECTOR (3 downto 0); -- 4-bit Giriş B

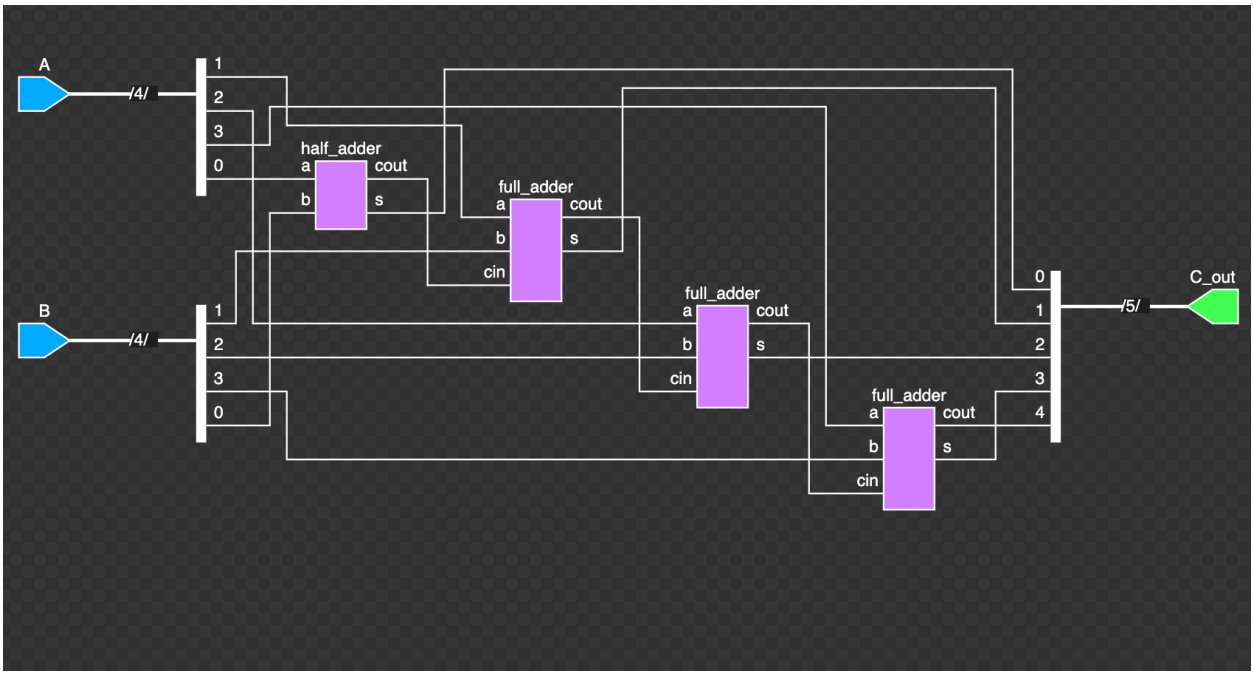
C_out : out STD_LOGIC_VECTOR (4 downto 0)); -- 5-bit Sonuç (C4 S3 S2 S1 S0)

```
end RCA_4BIT_STRUCTURAL;

architecture Structural of RCA_4BIT_STRUCTURAL is

    -- BİLEŞEN BEYANLARI (HA ve FA'yı kullanacağımızı belirtiyoruz)
    component HALF_ADDER
    Port ( A : in STD_LOGIC; B : in STD_LOGIC; S : out STD_LOGIC; Cout : out STD_LOGIC);
end component;

component FULL_ADDER
Port ( A : in STD_LOGIC; B : in STD_LOGIC; Cin : in STD_LOGIC; S : out STD_LOGIC; Cout : out STD_LOGIC);
end component;

    -- Elde (Carry) sinyallerini HA/FA'lar arasında taşımak için iç sinyal tanımlamaları
```

```
signal C_internal : STD_LOGIC_VECTOR (2 downto 0); -- C0, C1, C2


begin
    -- 1. Aşama (Bit 0): Half Adder (HA)
    -- C_out0 --> C_internal(0)
    HA_0: HALF_ADDER
    port map (
        A => A(0),
        B => B(0),
        S => C_out(0),          -- Toplamın 0. biti
        Cout => C_internal(0)   -- C0 (FA1'in Cin'i)
    );

    -- 2. Aşama (Bit 1): Full Adder 1 (FA1)
    -- Cin1 --> C_internal(0), Cout1 --> C_internal(1)
    FA_1: FULL_ADDER
    port map (
```

```
    A => A(1),
    B => B(1),
    Cin => C_internal(0),   -- C0
    S => C_out(1),          -- Toplamın 1. biti
    Cout => C_internal(1)   -- C1 (FA2'nin Cin'i)
);

-- 3. Aşama (Bit 2): Full Adder 2 (FA2)
-- Cin2 --> C_internal(1), Cout2 --> C_internal(2)
FA_2: FULL_ADDER
port map (
    A => A(2),
    B => B(2),
    Cin => C_internal(1),   -- C1
    S => C_out(2),          -- Toplamın 2. biti
    Cout => C_internal(2)   -- C2 (FA3'ün Cin'i)
);
```

```
    -- 4. Aşama (Bit 3): Full Adder 3 (FA3)
    -- Cin3 --> C_internal(2), Cout3 --> C_out(4)
    FA_3: FULL_ADDER
    port map (
        A => A(3),
        B => B(3),
        Cin => C_internal(2),   -- C2
        S => C_out(3),          -- Toplamın 3. biti
        Cout => C_out(4)        -- C3 (En son elde, 5. bit C4'e atanır)
    );

end Structural;
```



RCA Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity RCA_4BIT_TB is
end RCA_4BIT_TB;

architecture Behavioral of RCA_4BIT_TB is
```

```
    component RCA_4BIT_STRUCTURAL
    Port ( A : in  STD_LOGIC_VECTOR (3 downto 0);
       B : in  STD_LOGIC_VECTOR (3 downto 0);
       C_out : out STD_LOGIC_VECTOR (4 downto 0));
end component;

signal A_TB, B_TB : STD_LOGIC_VECTOR (3 downto 0) :=
(others => '0');
signal C_out_TB : STD_LOGIC_VECTOR (4 downto 0);

constant Delay : time := 100 ns;

begin
    uut: RCA_4BIT_STRUCTURAL
    port map (
       A => A_TB,
       B => B_TB,
       C_out => C_out_TB
```

```
);

stim_proc: process
begin
    -- Test 1: 5 + 3 = 8 (0101 + 0011 = 01000)
    A_TB <= "0101";
    B_TB <= "0011";
    wait for Delay;

    -- Test 2: 15 + 1 = 16 (1111 + 0001 = 10000)
    A_TB <= "1111";
    B_TB <= "0001";
    wait for Delay;

    -- Test 3: 15 + 15 = 30 (1111 + 1111 = 11110)
    A_TB <= "1111";
    B_TB <= "1111";
    wait for Delay;
```

```
        wait;
    end process;
end Behavioral;
```